

# **Informe Comparativo de Protocolos TCP vs UDP y Ejecución Secuencial vs Threads en Java**

**Ricardo Hurtado**

**Jose Guerrero**

**Samuel Giraldo**

## **Introducción**

En el desarrollo de aplicaciones distribuidas y concurrentes es fundamental comprender el comportamiento de los diferentes protocolos de comunicación y modelos de ejecución. En este taller se realizaron pruebas utilizando sockets TCP y UDP, así como programas implementados de manera secuencial y mediante threads en Java.

El objetivo principal fue analizar las diferencias en rendimiento, confiabilidad y tiempo de ejecución entre:

- El protocolo TCP y el protocolo UDP.
- La ejecución secuencial y la ejecución concurrente mediante threads.

Las pruebas fueron realizadas utilizando los programas contenidos en el archivo entregado, desarrollados en Java y ejecutados desde consola.

## **1. Comparación entre TCP y UDP**

### **Objetivo**

Comparar el comportamiento de los protocolos TCP y UDP en aplicaciones cliente-servidor, evaluando aspectos como:

- Tiempo de respuesta.
- Confiabilidad.
- Velocidad de transmisión.
- Complejidad de conexión.

### **Desarrollo de la práctica**

Para esta prueba se utilizaron los siguientes archivos:

## **TCP**

serTCPsocket.java

cliTCPsocket.java

## **UDP**

serUDPsocket.java

cliUDPsocket.java

Primero se compiló cada uno de los programas utilizando:

```
javac *.java
```

Posteriormente se ejecutó el servidor y el cliente en terminales separadas para cada protocolo.

### **Funcionamiento de TCP**

El protocolo TCP establece una conexión previa entre cliente y servidor antes de transmitir información. Esto garantiza:

Entrega correcta de los datos.

Orden de los paquetes.

Control de errores.

Durante la ejecución se observó que la comunicación es estable y confiable, aunque el proceso de conexión genera un mayor tiempo de respuesta.

### **Funcionamiento de UDP**

El protocolo UDP envía los paquetes directamente sin establecer conexión previa. Esto permite:

Mayor velocidad de transmisión.

Menor consumo de recursos.

Menor latencia.

Sin embargo, UDP no garantiza que los paquetes lleguen correctamente ni en el orden esperado.

### **Resultados obtenidos**

| <b>Característica</b> | <b>TCP</b>           | <b>UDP</b>              |
|-----------------------|----------------------|-------------------------|
| Tipo de conexión      | Orientado a conexión | No orientado a conexión |
| Velocidad             | Media                | Alta                    |
| Confiabilidad         | Alta                 | Baja                    |
| Control de errores    | Sí                   | No                      |
| Orden de paquetes     | Garantizado          | No garantizado          |
| Complejidad           | Mayor                | Menor                   |

### **Análisis de resultados**

Durante las pruebas se evidenció que TCP ofrece una comunicación más segura y estable debido al control de errores y confirmación de recepción. Esto lo hace ideal para aplicaciones donde la integridad de la información es prioritaria, como páginas web, transferencia de archivos y bases de datos.

Por otro lado, UDP presentó una transmisión más rápida debido a la ausencia de mecanismos de verificación y establecimiento de conexión. Este comportamiento lo hace adecuado para aplicaciones en tiempo real como videojuegos, videollamadas y streaming.

Aunque UDP es más eficiente en velocidad, existe el riesgo de pérdida de paquetes, mientras que TCP prioriza la confiabilidad sobre el rendimiento.

## **2. Comparación entre Ejecución Secuencial y Threads**

### **Objetivo**

Analizar las diferencias entre la ejecución secuencial y la ejecución concurrente utilizando threads en Java, evaluando principalmente:

- Tiempo total de ejecución.
- Aprovechamiento del procesamiento concurrente.
- Eficiencia del sistema.

### **Desarrollo de la práctica**

Para esta prueba se utilizaron los siguientes archivos:

- Main.java
- MainThread.java
- MainRunnable.java
- Cajera.java
- CajeraThread.java

Los programas fueron compilados utilizando:

```
javac *.java  
time java -cp bin tallerThreads.Main  
time java -cp bin tallerThreads.MainThread  
time java -cp bin tallerThreads.MainRunnable
```

Posteriormente se ejecutaron los diferentes programas para observar el comportamiento de cada modelo.

### **Ejecución Secuencial**

En la ejecución secuencial las tareas se realizan una después de otra. Cada proceso debe finalizar antes de que el siguiente pueda comenzar.

Este modelo es sencillo de implementar, pero puede aumentar considerablemente el tiempo total de ejecución cuando existen múltiples tareas.

Durante las pruebas se observó que:

- Los procesos esperaban turno.
- El tiempo total era mayor.
- El uso del procesador era menos eficiente.

### **Ejecución con Threads**

La implementación con threads permitió ejecutar múltiples tareas de manera concurrente.

Cada thread ejecutó una tarea independiente, permitiendo que varios procesos se desarrollaran simultáneamente.

Durante las pruebas se observó:

- Reducción del tiempo total de ejecución.
- Mayor eficiencia.
- Procesamiento concurrente de tareas.

## Resultados obtenidos

| Característica       | Secuencial | Threads               |
|----------------------|------------|-----------------------|
| Ejecución simultánea | No         | Sí                    |
| Tiempo total         | Mayor      | Menor                 |
| Complejidad          | Baja       | Media                 |
| Uso del procesador   | Menor      | Mayor aprovechamiento |
| Rendimiento          | Menor      | Mayor                 |

## Análisis de resultados

La implementación secuencial resultó más lenta debido a que cada tarea debía finalizar antes de iniciar la siguiente. Este comportamiento aumenta el tiempo total de procesamiento cuando existen múltiples operaciones.

En contraste, el uso de threads permitió ejecutar varias tareas al mismo tiempo, reduciendo considerablemente el tiempo total de ejecución y mejorando el rendimiento general del sistema.

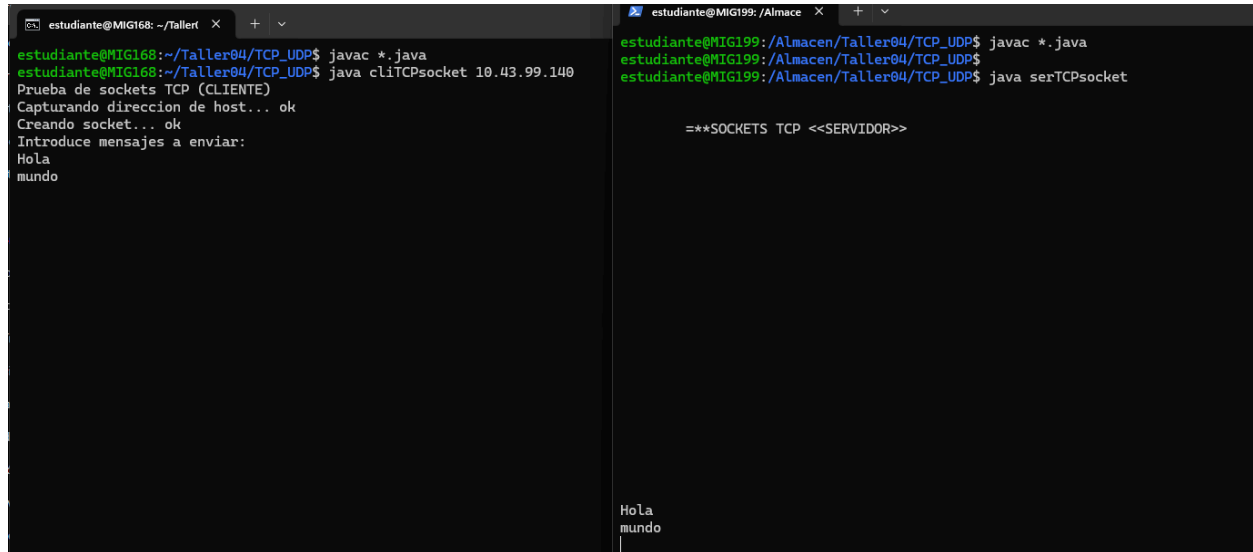
Sin embargo, el uso de threads también incrementa la complejidad del programa, ya que es necesario controlar adecuadamente la sincronización y el manejo de recursos compartidos.

## Conclusiones

- TCP proporciona una comunicación más confiable y segura, aunque con mayor consumo de tiempo y recursos.
- UDP ofrece mayor velocidad y menor latencia, pero no garantiza la entrega correcta de los datos.
- La ejecución secuencial es más simple de implementar, pero menos eficiente cuando existen múltiples tareas.

- Los threads permiten mejorar significativamente el rendimiento mediante procesamiento concurrente.
- La elección entre TCP o UDP, así como entre ejecución secuencial o concurrente, depende de las necesidades específicas de cada aplicación.

## 1. TCP



The image shows two terminal windows side-by-side. The left window, titled 'estudiante@MIG168: ~/Taller', shows the execution of a client program. The right window, titled 'estudiante@MIG199: /Almacen', shows the execution of a server program. The client sends the message 'Hola mundo' and the server receives it, displaying 'HOLA mundo'.

```

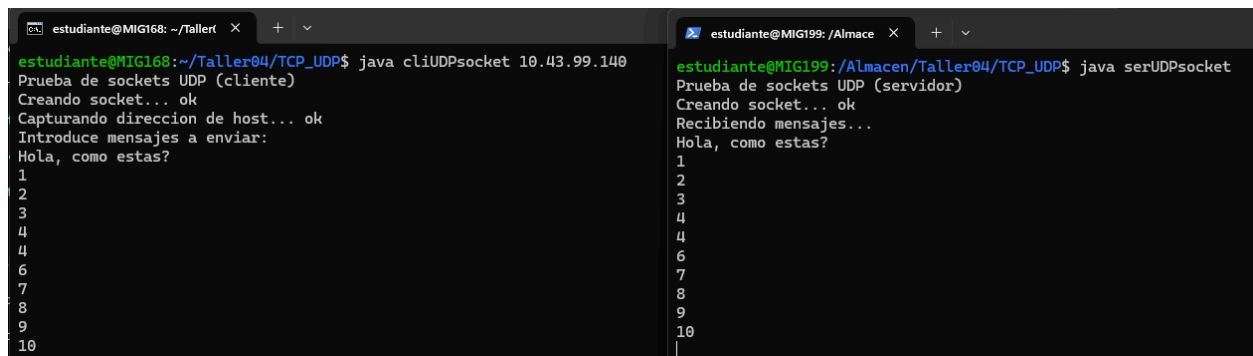
estudiante@MIG168: ~/Taller
estudiante@MIG168:~/Taller04/TCP_UDP$ javac *.java
estudiante@MIG168:~/Taller04/TCP_UDP$ java cliTCPsocket 10.43.99.140
Prueba de sockets TCP (CLIENTE)
Capturando direccion de host... ok
Creando socket... ok
Introduce mensajes a enviar:
Hola
mundo

estudiante@MIG199: /Almacen
estudiante@MIG199:/Almacen/Taller04/TCP_UDP$ javac *.java
estudiante@MIG199:/Almacen/Taller04/TCP_UDP$ java serTCPsocket

==SOCKETS TCP <<SERVIDOR>>

HOLA
mundo
  
```

## 2. UDP



The image shows two terminal windows side-by-side. The left window, titled 'estudiante@MIG168: ~/Taller', shows the execution of a client program. The right window, titled 'estudiante@MIG199: /Almacen', shows the execution of a server program. The client sends a series of numbers (1-10) and the server receives them, displaying 'HOLA, como estas?' followed by the numbers 1-10.

```

estudiante@MIG168: ~/Taller
estudiante@MIG168:~/Taller04/TCP_UDP$ java cliUDPsocket 10.43.99.140
Prueba de sockets UDP (cliente)
Creando socket... ok
Capturando direccion de host... ok
Introduce mensajes a enviar:
Hola, como estas?
1
2
3
4
4
6
7
8
9
10

estudiante@MIG199: /Almacen
estudiante@MIG199:/Almacen/Taller04/TCP_UDP$ java serUDPsocket
Prueba de sockets UDP (servidor)
Creando socket... ok
Recibiendo mensajes...
HOLA, como estas?
1
2
3
4
4
6
7
8
9
10
  
```

## 1.Main

```
estudiante@MIG185:/Almacen/Taller04/THREADS$ time java -cp bin tallerThreads.Main
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 15seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 16seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 19seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 24seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 25seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 26seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 26seg

real    0m26,114s
user    0m0,153s
sys     0m0,069s
```

## 2.MainThread

```
estudiante@MIG185:/Almacen/Taller04/THREADS$ time java -cp bin tallerThreads.MainThread
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 1seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 9seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 11seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 11seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg

real    0m15,101s
user    0m0,172s
sys     0m0,070s
```

## 3.MainRunnable

```
estudiante@MIG185:/Almacen/Taller04/THREADS$ time java -cp bin tallerThreads.MainRunnable
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 1seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 9seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 11seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 11seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg

real    0m15,091s
user    0m0,166s
sys     0m0,058s
```